

ROADMAP TRIỂN KHAI DỰ ÁN CÁ NHÂN

CV Screening & Ranking Agent

Agent chấm điểm và xếp hạng hàng loạt CV theo JD cho nhà tuyển dụng, ứng dụng LangChain, LangGraph, Guardrails, Evaluation và LLM Gateway

Tài liệu hướng dẫn triển khai từng giai đoạn (không kèm code)

Dùng cho mục đích xây dựng dự án cá nhân đưa vào Resume

1. Mô tả chi tiết dự án

1.1 Bài toán

Nhà tuyển dụng khi nhận số lượng lớn CV cho một vị trí thường mất nhiều thời gian đọc thủ công để tìm ra ứng viên phù hợp. Dự án này xây dựng một agent nhận vào một JD (Job Description) và một batch CV, tự động chấm điểm mức độ phù hợp của từng CV so với JD, rồi trả về một danh sách ứng viên đã được xếp hạng kèm lý giải rõ ràng cho từng điểm số — giúp nhà tuyển dụng rút ngắn thời gian sàng lọc ban đầu mà vẫn giữ quyền quyết định cuối cùng.

1.2 Giá trị mang lại

Để dự án không dừng ở mức "so khớp CV-JD bằng LLM" đơn thuần (vốn đã có rất nhiều người làm), dự án tập trung vào các khía cạnh làm cho một hệ thống chấm điểm hàng loạt có thể tin cậy được trong thực tế:

- **Minh bạch trong cách chấm điểm:** điểm số không phải một con số "hộp đen" do một lần gọi LLM sinh ra, mà được phân rã thành nhiều subagent chấm theo từng khía cạnh riêng (kỹ năng, kinh nghiệm, học vấn), giúp nhà tuyển dụng biết chính xác vì sao một CV được điểm cao/thấp.
- **Chống bịa đặt (hallucination) khi liệt kê kỹ năng khớp:** guardrail đối chiếu lại với CV gốc để đảm bảo agent không tự "thêm" kỹ năng ứng viên không thực sự có — đây là rủi ro thường gặp và ảnh hưởng trực tiếp tới độ tin cậy của kết quả xếp hạng.
- **Con người vẫn giữ quyền quyết định:** agent chỉ tự động xếp hạng, các trường hợp điểm số nằm ở vùng không chắc chắn được đẩy sang cho người dùng duyệt thay vì để agent tự quyết hoàn toàn.
- **Độ chính xác được đo bằng số liệu khách quan** thay vì chỉ "nhìn có vẻ đúng" — tận dụng chính nhãn ngành nghề có sẵn trong bộ dữ liệu Kaggle làm ground truth để tính độ chính xác thật (chi tiết ở Mục 1.3 và Giai đoạn 7).
- **Tối ưu chi phí khi xử lý số lượng lớn CV** bằng LLM Gateway — vì bài toán chấm hàng loạt CV có tính lặp lại cao, đây là nơi cost tracking, caching và routing model tạo giá trị thực tế.

Về mặt kiến trúc áp dụng: Structured Output, Tools, Message Types (LangChain), LangGraph orchestration + Human-in-the-loop (áp dụng tư duy subagent học từ khoá Deep Agents), Guardrails before/after-agent, Evaluation (LLM-as-judge + độ chính xác định lượng), và LLM Gateway (cost tracking, caching, fallback, routing).

1.3 Phạm vi dữ liệu (phù hợp sinh viên, không cần dữ liệu nội bộ doanh nghiệp)

- **Nguồn CV:** bộ dữ liệu công khai trên Kaggle, ví dụ "Resume Dataset" — bộ này có sẵn cột **Category** (ví dụ: Java Developer, Data Science, HR, Business Analyst...). Cột này ban đầu không dùng để chấm điểm, nhưng sẽ được giữ lại riêng để làm **ground truth cho việc đánh giá độ chính xác** ở Giai đoạn 7 — một lợi thế quan trọng vì bạn không cần tự gán nhãn thủ công hàng chục CV, điều mà hầu hết đồ án sinh viên khác bỏ qua vì tốn công.

- **Nguồn JD:** tự soạn 3-5 JD mẫu ứng với một số Category có sẵn trong dataset (ví dụ soạn JD cho vị trí "Java Developer" để đối chiếu với các CV có Category tương ứng), hoặc lấy JD thật công khai từ các trang tuyển dụng rồi map thủ công sang Category gần nhất.

1.4 Input / Output tổng thể của hệ thống

Input	(1) Một file JD dạng văn bản mô tả vị trí tuyển dụng. (2) Một batch CV (nhiều file, văn bản/PDF) lấy từ bộ dữ liệu Kaggle.
Output	(1) Một danh sách ứng viên hoàn chỉnh đã xếp hạng theo mức độ phù hợp, mỗi ứng viên kèm: điểm tổng hợp, điểm từng khía cạnh (kỹ năng/kinh nghiệm/học vấn), kỹ năng khớp/thiếu, khuyến nghị, và lý giải — xuất ra dạng bảng (CSV/JSON) và hiển thị trên giao diện. (2) Danh sách con các ứng viên "cần duyệt thủ công" (điểm nằm ở vùng chưa chắc chắn). (3) Báo cáo evaluation: độ chính xác của agent so với nhãn Category thật, và điểm chất lượng lý giải do LLM-as-judge chấm. (4) Báo cáo chi phí: tổng token, cost theo model, hiệu quả của việc route/cache model khi xử lý hàng loạt CV.

1.5 Kiến trúc tổng quan (luồng xử lý)

JD + batch CV → Tiền xử lý dữ liệu → Che thông tin nhạy cảm (before-agent guardrail) → Subagent chấm điểm theo từng khía cạnh (kỹ năng / kinh nghiệm / học vấn) → Tổng hợp kết quả (orchestrator) → Kiểm tra output có bịa đặt kỹ năng không thực sự tồn tại trong CV (after-agent guardrail) → Nếu điểm nằm ở vùng biên → chuyển cho người dùng duyệt (human-in-the-loop) → Trả kết quả structured output cho từng CV → Tổng hợp toàn bộ batch thành một danh sách xếp hạng hoàn chỉnh.

2. Cấu trúc thư mục dự án

Cấu trúc được tổ chức theo nguyên tắc tách biệt rõ: dữ liệu, schema, logic subagent, guardrail, graph điều phối, gateway, evaluation, và giao diện demo.

```
cv-screening-agent/
├── .env.example           # khai báo API key mẫu (OpenAI/Groq/Gemini...)
├── pyproject.toml         # quản lý bằng UV, khai báo dependencies
├── uv.lock
├── README.md             # mô tả dự án, hướng dẫn chạy, kết quả evaluation
├── data/
│   ├── raw/              # CV gốc tải từ Kaggle (kèm cột Category), JD mẫu
│   └── processed/         # CV đã làm sạch, chuẩn hoá text, giữ lại Category riêng
├── src/
│   ├── config.py         # load biến môi trường, cấu hình model mặc định
│   ├── schemas/
│   │   ├── cv_schema.py  # Pydantic model cho kết quả chấm điểm 1 CV
│   │   └── eval_schema.py # schema cho dữ liệu evaluation
│   ├── data_processing/
│   │   ├── loader.py     # đọc CV/JD từ data/raw
│   │   └── cleaner.py    # chuẩn hoá text, tách section trong CV
│   ├── subagents/
│   │   ├── skill_matcher.py # subagent so khớp kỹ năng CV với JD
│   │   ├── experience_scorer.py # subagent đánh giá kinh nghiệm làm việc
│   │   ├── education_scorer.py # subagent đánh giá học vấn
│   │   └── orchestrator.py  # tổng hợp kết quả từ 3 subagent thành 1 kết luận
│   ├── guardrails/
│   │   ├── pii_redaction.py # before-agent: che tên/tuổi/giới tính trước khi chấm
│   │   ├── hallucination_checker.py # after-agent: phát hiện kỹ năng bịa đặt không có trong CV
│   │   └── guardrail_pipeline.py # ghép các guardrail thành pipeline nhiều lớp
│   ├── graph/
│   │   ├── state.py      # định nghĩa State object cho LangGraph
│   │   ├── nodes.py      # các node: redact, score, guardrail, hitl, output
│   │   └── build_graph.py # khai báo graph, cạnh nối, điều kiện rẽ nhánh
│   ├── gateway/
│   │   ├── litellm_config.py # khai báo danh sách model + gateway routing
│   │   ├── router.py        # logic chọn model theo độ phức tạp subagent
│   │   └── cost_tracker.py   # ghi log token/cost mỗi lần gọi model
│   └── evaluation/
│       └── eval_dataset_builder.py # chọn tập CV mẫu, dùng cột Category có sẵn làm ground truth
```

```
| | | llm_judge.py          # dùng LLM chấm chất lượng reasoning của agent
| | | accuracy_metrics.py  # tính độ chính xác matching so với Category thật
| | | run_evaluation.py    # script chạy toàn bộ evaluation, xuất report
| | |
| | | └─ utils/
| | |     └─ logger.py      # logging chung cho toàn dự án
| | |
| └─ app/
|     └─ streamlit_app.py  # giao diện demo: upload JD + batch CV, xem bảng xếp hạng, duy
|
| └─ tests/
|     └─ test_schemas.py
|     └─ test_guardrails.py
|     └─ test_graph.py
|
| └─ reports/
|     └─ evaluation_report.md # độ chính xác + chất lượng lý giải + chi phí xử lý batch
|
| └─ notebooks/
|     └─ exploration.ipynb  # khám phá dữ liệu ban đầu, thử nghiệm nhanh
```

3. Chi tiết từng giai đoạn triển khai

Mỗi giai đoạn được trình bày theo format: **Mục tiêu** → **File cần tạo & chức năng** → **Input/Output của giai đoạn** → **Thư viện liên quan** → **Lưu ý kỹ thuật**. Các giai đoạn nên được làm tuần tự vì giai đoạn sau phụ thuộc vào output của giai đoạn trước.

Giai đoạn 0 — Khởi tạo môi trường & cấu trúc dự án

Mục tiêu: Thiết lập môi trường phát triển sạch, có thể tái lập, và khung thư mục ban đầu trước khi viết bất kỳ logic nào.

File / việc cần làm và chức năng

- `pyproject.toml` — khai báo toàn bộ dependencies của dự án, khởi tạo bằng UV thay vì pip thông thường để quản lý virtual environment nhanh và nhất quán.
- `.env.example` — liệt kê các biến môi trường cần thiết (API key cho từng provider model) mà không chứa giá trị thật, để người khác clone project biết cần khai báo gì.
- `src/config.py` — chức năng đọc biến môi trường, cung cấp một điểm cấu hình trung tâm cho toàn bộ project (model mặc định, ngưỡng rủi ro, đường dẫn dữ liệu).
- Tạo toàn bộ khung thư mục rỗng theo cấu trúc ở Mục 2 để các giai đoạn sau chỉ việc điền logic vào đúng vị trí.

Input	Không có (giai đoạn khởi tạo)
Output	Một project skeleton chạy được lệnh cài đặt, virtual environment sẵn sàng, cấu trúc thư mục đầy đủ

Thư viện / công cụ	Vai trò
uv	Tạo và quản lý virtual environment, cài dependency nhanh hơn pip
python-dotenv	Đọc biến môi trường từ file <code>.env</code> vào <code>config.py</code>

Giai đoạn 1 — Thu thập & xử lý dữ liệu CV, JD

Mục tiêu: Có một tập dữ liệu CV và JD sạch, ở định dạng văn bản thuần, sẵn sàng để đưa vào agent chấm điểm — đồng thời giữ lại cột Category gốc để dùng riêng cho việc đánh giá độ chính xác sau này.

File cần tạo và chức năng

- `data_processing/loader.py` — chức năng đọc file CV (txt/pdf/csv tùy định dạng dataset Kaggle) và JD từ thư mục `data/raw`, trả về dạng object Python thống nhất (id, nội dung, Category gốc nếu có, metadata khác).
- `data_processing/cleaner.py` — chức năng chuẩn hoá văn bản (loại bỏ ký tự thừa, xuống dòng lộn xộn), và cố gắng tách CV thành các section rõ ràng (học vấn, kinh nghiệm, kỹ năng) để các subagent ở giai đoạn sau dễ xử lý hơn.
- Lưu kết quả đã làm sạch vào `data/processed` ở định dạng nhất quán (ví dụ JSON có field cố định, trong đó Category được lưu riêng một field không đưa vào phần nội dung agent sẽ đọc) để các giai đoạn sau chỉ cần đọc từ đây, không cần đọc lại raw data.

Input	Bộ CV thô tải từ Kaggle (định dạng gốc của dataset, có cột Category) + JD tự soạn
Output	File JSON/CSV chứa CV và JD đã chuẩn hoá, có cấu trúc field rõ ràng, lưu tại <code>data/processed</code>

Thư viện	Vai trò
pandas	Đọc/ghi CSV, thao tác dữ liệu dạng bảng
pdfplumber hoặc pypdf	Trích xuất text nếu CV ở định dạng PDF
re (built-in)	Làm sạch text bằng regex

Lưu ý: Category chỉ được dùng nội bộ cho việc evaluate ở Giai đoạn 7, không được đưa vào input mà agent nhìn thấy khi chấm điểm — nếu không, agent có thể "ăn gian" bằng cách dựa vào chính Category để suy ra điểm thay vì thực sự đọc nội dung CV.

Giai đoạn 2 — Thiết kế Structured Output Schema

Mục tiêu: Định nghĩa trước hình dạng dữ liệu mà agent phải trả về, đảm bảo output luôn có cấu trúc cố định để dễ tổng hợp thành danh sách xếp hạng, dễ evaluate.

File cần tạo và chức năng

- `schemas/cv_schema.py` — định nghĩa bằng Pydantic model cấu trúc kết quả chấm điểm một CV: điểm phù hợp tổng thể, điểm từng khía cạnh (kỹ năng/kinh nghiệm/học vấn), danh sách kỹ năng khớp, danh sách kỹ năng thiếu, khuyến nghị (shortlist/reject/borderline), và phần lý giải (reasoning) dạng text. Đây là schema chính agent phải tuân theo khi trả kết quả.
- `schemas/eval_schema.py` — định nghĩa cấu trúc dữ liệu cho một bản ghi evaluation: gồm CV đầu vào, câu trả lời agent, Category thật (ground truth), và điểm số do LLM-as-judge chấm.

Input	Không có — đây là bước thiết kế contract dữ liệu, chưa chạy logic thật
Output	Các class schema có thể import và tái sử dụng xuyên suốt toàn bộ dự án

Thư viện	Vai trò
pydantic	Định nghĩa schema chính cho structured output của agent, có validation tự động

Gợi ý nâng cao (không bắt buộc): có thể thử định nghĩa lại cùng một schema bằng `TypedDict` và `dataclass` ở một file riêng chỉ để so sánh, rồi ghi nhận trong README lý do bạn chọn Pydantic cho production code (có validation, dễ tích hợp với LangChain structured output) — đây là cách thể hiện bạn hiểu rõ sự khác biệt giữa 3 cách đã học thay vì chỉ biết một cách.

Giai đoạn 3 — Xây dựng các Subagent chấm điểm (tư duy Deep Agents, hiện thực bằng LangGraph)

Mục tiêu: Thay vì dùng một agent duy nhất chấm điểm toàn bộ CV cùng lúc (dễ bỏ sót, khó audit, kết quả như "hộp đen"), chia nhỏ bài toán thành các subagent chuyên trách từng khía cạnh, sau đó một orchestrator tổng hợp lại. Ở bước này mỗi subagent chỉ là một hàm/thành phần độc lập; việc điều phối chúng thành một luồng hoàn chỉnh sẽ do LangGraph đảm nhiệm ở Giai đoạn 5 — **framework duy nhất cho toàn bộ phần orchestration của dự án là LangGraph**, khái niệm subagent chỉ là tư duy thiết kế vay mượn từ khoá Deep Agents.

File cần tạo và chức năng

- `subagents/skill_matcher.py` — subagent nhận JD + phần kỹ năng của CV, trả về danh sách kỹ năng khớp/thiếu và điểm số riêng cho khía cạnh kỹ năng.
- `subagents/experience_scorer.py` — subagent đánh giá số năm kinh nghiệm và mức độ liên quan của các vị trí đã làm so với JD, trả điểm riêng cho khía cạnh kinh nghiệm.
- `subagents/education_scorer.py` — subagent đánh giá học vấn có đáp ứng yêu cầu tối thiểu của JD hay không (bằng cấp/ngành học).
- `subagents/orchestrator.py` — nhận kết quả từ 3 subagent trên, tổng hợp thành một điểm số cuối cùng và một đoạn lý giải duy nhất theo đúng `cv_schema.py` đã định nghĩa ở Giai đoạn 2.

Input	1 CV đã làm sạch (từ <code>data/processed</code>) + 1 JD
Output	Một object tuân theo <code>cv_schema.py</code> , chứa điểm tổng hợp và lý giải

Thư viện	Vai trò
langchain	Model Integration — kết nối tới LLM provider cho từng subagent
langchain-core (Tools)	Định nghĩa các hàm trích xuất (ví dụ đếm năm kinh nghiệm) thành tool mà subagent có thể gọi
langchain-core (Message Types)	Quản lý hội thoại nội bộ giữa orchestrator và từng subagent

Giai đoạn 4 — Xây dựng Guardrails nhiều lớp

Mục tiêu: Đảm bảo hệ thống không rò rỉ thông tin cá nhân không cần thiết trước khi chấm điểm, và không sinh ra kết luận bịa đặt (liệt kê kỹ năng ứng viên không thực sự có) sau khi chấm điểm.

File cần tạo và chức năng

- `guardrails/pii_redaction.py` — guardrail dạng **before-agent**: quét CV trước khi đưa vào subagent, che (redact) các thông tin cá nhân không cần thiết cho việc đánh giá (số điện thoại, email, ảnh nếu có), chỉ giữ lại phần nội dung chuyên môn liên quan.
- `guardrails/hallucination_checker.py` — guardrail dạng **after-agent**: đối chiếu danh sách "kỹ năng khớp" mà agent liệt kê với nội dung CV gốc, phát hiện và cảnh báo trường hợp agent liệt kê kỹ năng không thực sự xuất hiện trong CV — đây là guardrail quan trọng nhất của dự án vì ảnh hưởng trực tiếp tới độ tin cậy của kết quả xếp hạng.
- `guardrails/guardrail_pipeline.py` — ghép các guardrail trên thành một pipeline tuần tự (redact → chấm điểm → kiểm tra hallucination), có thể bật/tắt từng lớp để so sánh hiệu quả.

Input	CV thô (cho before-agent) và kết quả structured output từ orchestrator (cho after-agent)
Output	CV đã redact (đưa vào subagent), và một flag "pass/fail" kèm lý do nếu output vi phạm guardrail

Thư viện	Vai trò
langchain (PII Detection Middleware)	Middleware dựng sẵn để phát hiện và che thông tin cá nhân
presidio-analyzer / presidio-anonymizer (tuỳ chọn thay thế)	Thư viện chuyên biệt phát hiện PII nếu middleware built-in chưa đủ mạnh cho tên riêng
rapidfuzz	So khớp gần đúng để kiểm tra kỹ năng "hallucinate" có thực sự xuất hiện trong CV gốc không

Giai đoạn 5 — Điều phối bằng LangGraph & Human-in-the-loop

Mục tiêu: Kết nối toàn bộ các thành phần (redact → subagent → guardrail → human review nếu cần) thành một luồng xử lý có trạng thái rõ ràng, có thể theo dõi từng bước, và cho phép con người can thiệp đúng lúc, chạy được cho cả một batch nhiều CV.

File cần tạo và chức năng

- `graph/state.py` — định nghĩa cấu trúc State đi xuyên suốt graph: CV gốc, CV đã redact, kết quả từng subagent, kết quả guardrail, quyết định cuối cùng, trạng thái đã được người duyệt hay chưa.
- `graph/nodes.py` — định nghĩa từng node tương ứng với một bước xử lý: node redact, node gọi 3 subagent (có thể chạy song song), node orchestrator tổng hợp, node guardrail after-agent, node quyết định có cần human-in-the-loop hay không, node trả kết quả cuối.
- `graph/build_graph.py` — khai báo graph hoàn chỉnh: nối các node theo đúng thứ tự, định nghĩa cạnh có điều kiện (nếu điểm nằm trong khoảng borderline thì rẽ sang node chờ người duyệt thay vì đi thẳng tới node trả kết quả), và cách graph này được gọi lặp lại cho từng CV trong batch rồi gộp kết quả thành danh sách xếp hạng.

Input	1 batch CV + 1 JD (điểm bắt đầu của graph, chạy cho từng CV trong batch)
Output	Kết quả cuối cùng cho từng CV, kèm trạng thái "đã duyệt tự động" hoặc "chờ người duyệt"; tổng hợp thành danh sách xếp hạng toàn batch

Thư viện	Vai trò
langgraph	Xây dựng state graph, quản lý luồng điều phối giữa các node
langgraph (Human-in-the-loop / interrupt)	Tạm dừng graph tại node borderline để chờ input từ người dùng
langgraph (Streaming)	Trả kết quả từng CV theo thời gian thực khi xử lý cả batch, hữu ích khi demo trên giao diện với batch lớn

Ngưỡng rẽ nhánh borderline: bạn cần tự định nghĩa một khoảng điểm số (ví dụ 40-60 trên thang 100) coi là "chưa chắc chắn" — đây là phần thiết kế logic bạn tự quyết định và nên giải thích rõ lý do lựa chọn trong README.

Giai đoạn 6 — Tích hợp LLM Gateway (routing, cost, cache, fallback)

Mục tiêu: Tối ưu chi phí và độ ổn định khi gọi model cho nhiều subagent và nhiều CV trong cùng một batch, thay vì gọi cứng một model duy nhất cho mọi tác vụ.

File cần tạo và chức năng

- `gateway/litellm_config.py` — khai báo danh sách các model sẽ dùng (ví dụ một model free-tier cho subagent đơn giản như `education_scorer`, một model mạnh hơn cho orchestrator tổng hợp), cấu hình theo chuẩn của gateway.
- `gateway/router.py` — logic quyết định subagent nào gọi model nào (smart routing), ví dụ dựa trên độ dài input hoặc độ phức tạp của tác vụ.
- `gateway/cost_tracker.py` — ghi lại số token và chi phí ước tính của mỗi lần gọi model, tổng hợp theo subagent, theo batch CV đã xử lý, phục vụ cho báo cáo chi phí cuối cùng.

Input	Prompt/request từ từng subagent trong Giai đoạn 3, lặp lại cho toàn bộ batch CV
Output	Response từ model được chọn, kèm log chi phí/latency được ghi lại theo từng CV; tự động fallback sang model dự phòng nếu model chính lỗi; cache kết quả cho các request trùng lặp trong batch

Thư viện	Vai trò
litellm	Lớp gateway thống nhất để gọi nhiều LLM provider khác nhau qua một interface duy nhất
litellm (Caching)	Cache lại kết quả cho các request giống hệt nhau, giảm chi phí khi test lặp lại hoặc khi batch có CV tương tự nhau
litellm (Fallback / Router)	Tự động chuyển sang model dự phòng khi model chính timeout hoặc lỗi giữa chừng batch lớn

Giai đoạn 7 — Evaluation: Đo độ chính xác Matching bằng dữ liệu thật

Mục tiêu: Chứng minh bằng số liệu khách quan rằng agent chấm điểm CV-JD chính xác, không chỉ "nhìn có vẻ hợp lý" — đây là phần tạo giá trị khác biệt của dự án khi trình bày trong resume/phỏng vấn, vì hầu hết CV matcher tự chế đều không có bước kiểm chứng độ chính xác.

File cần tạo và chức năng

- `evaluation/eval_dataset_builder.py` — chọn ra một tập con CV (khoảng 30-50 CV) từ nhiều Category khác nhau trong dataset Kaggle. Vì mỗi CV đã có sẵn nhãn Category thật (ví dụ "Java Developer", "HR", "Data Science"), bạn không cần tự gán nhãn thủ công: với một JD cho vị trí X, ground truth hợp lý là CV thuộc đúng Category X phải được chấm điểm cao hơn CV thuộc Category hoàn toàn không liên quan (ví dụ JD "Java Developer" thì CV Category "Arts" phải bị chấm thấp).
- `evaluation/accuracy_metrics.py` — chạy JD mẫu qua toàn bộ tập CV đã chọn, tính các chỉ số: có bao nhiêu % CV đúng Category được xếp vào nhóm điểm cao (precision@k), điểm trung bình của nhóm CV đúng Category so với nhóm CV không liên quan có cách biệt rõ ràng hay không.
- `evaluation/llm_judge.py` — dùng một model độc lập đóng vai "giám khảo" để chấm chất lượng phần lý giải (reasoning) của agent: lý giải có logic, có bám sát nội dung CV hay không, theo thang điểm cố định — bổ sung cho accuracy_metrics vì đo chất lượng lời giải thích chứ không chỉ con số điểm.
- `evaluation/run_evaluation.py` — script chạy toàn bộ evaluation (accuracy theo Category, điểm LLM-judge, thống kê chi phí từ Giai đoạn 6), xuất kết quả ra `reports/evaluation_report.md`.

Input	Tập CV có Category thật (ground truth có sẵn từ dataset) + JD mẫu tương ứng + agent pipeline hoàn chỉnh
Output	Một báo cáo markdown: độ chính xác matching theo Category, điểm chất lượng lý giải, và thống kê chi phí xử lý batch

Thư viện	Vai trò
langchain / langsmith (tùy chọn)	Hỗ trợ tạo bộ dữ liệu evaluation và chạy LLM-as-judge có sẵn pattern
pandas	Tính precision@k, tổng hợp bảng kết quả theo Category
scikit-learn (tùy chọn)	Tính thêm các chỉ số ranking chuẩn (ví dụ AUC) nếu muốn báo cáo chuyên sâu hơn
matplotlib hoặc plotly	Vẽ biểu đồ phân bố điểm số giữa nhóm CV đúng Category và nhóm không liên quan

Giai đoạn 8 — Giao diện Demo

Mục tiêu: Có một giao diện trực quan để người xem (nhà tuyển dụng, người phỏng vấn) có thể tự trải nghiệm sản phẩm mà không cần đọc code.

File cần tạo và chức năng

- `app/streamlit_app.py` — giao diện cho phép: (1) upload 1 JD và một batch CV (nhiều file cùng lúc), (2) hiển thị bảng xếp hạng ứng viên đầy đủ theo structured output, có thể sắp xếp theo điểm và xuất ra file CSV, (3) khu vực "Cần duyệt" hiển thị các CV borderline đang chờ human-in-the-loop, cho phép người dùng bấm Approve/Reject trực tiếp trên giao diện, (4) một tab "Đánh giá hệ thống" hiển thị kết quả từ `evaluation_report.md` (độ chính xác, chi phí).

Input	File JD/CV do người dùng upload qua giao diện
Output	Giao diện web hiển thị bảng xếp hạng đầy đủ, khu vực duyệt thủ công, và tab báo cáo đánh giá hệ thống

Thư viện	Vai trò
streamlit	Xây dựng giao diện web nhanh cho demo, không cần biết frontend chuyên sâu
plotly (tích hợp với streamlit)	Biểu đồ tương tác cho tab đánh giá hệ thống ngay trên giao diện

Giai đoạn 9 — Kiểm thử, tài liệu hoá & đóng gói cho Resume

Mục tiêu: Hoàn thiện dự án ở mức có thể trình bày chuyên nghiệp: có test, có README rõ ràng, có số liệu cụ thể để đưa vào resume.

Việc cần làm

- `tests/test_schemas.py`, `test_guardrails.py`, `test_graph.py` — viết test cơ bản đảm bảo schema luôn validate đúng, guardrail chặn đúng trường hợp vi phạm đã biết trước, và graph chạy hết luồng không bị lỗi kể cả với batch nhiều CV.
- `README.md` — mô tả bài toán, kiến trúc (có thể vẽ sơ đồ), hướng dẫn cài đặt/chạy, và quan trọng nhất: trích dẫn số liệu cụ thể từ `reports/evaluation_report.md` (ví dụ: "đạt X% precision@5 khi xếp hạng CV đúng Category lên top", "tiết kiệm Y% chi phí xử lý batch nhờ caching và smart routing").
- Chuẩn bị 1 đoạn mô tả ngắn (2-3 câu) dùng riêng cho resume, nêu bật: bài toán giải quyết, kỹ thuật chính đã dùng, và một con số kết quả đo được.

Thư viện	Vai trò
pytest	Chạy các test đã viết, đảm bảo project ổn định trước khi công khai lên GitHub

4. Bảng tổng hợp toàn bộ thư viện sử dụng

Thư viện	Nhóm chức năng	Dùng ở giai đoạn
uv	Quản lý môi trường	0
python-dotenv	Cấu hình	0
pandas	Xử lý dữ liệu bảng	1, 7
pdfplumber / pypdf	Đọc CV dạng PDF	1
pydantic	Structured Output	2, 3
langchain	Model Integration, Tools, Message Types (dùng cho từng subagent độc lập)	3
langchain PII Middleware / presidio	Guardrail before-agent	4
rapidfuzz	Kiểm tra hallucination kỹ năng	4
langgraph	Orchestration, Human-in-the-loop, Streaming	5
litellm	LLM Gateway: routing, cost tracking, cache, fallback	6
scikit-learn	Chỉ số ranking chuyên sâu (tùy chọn)	7
matplotlib / plotly	Trực quan hoá kết quả đánh giá & demo	7, 8
streamlit	Giao diện demo	8
pytest	Kiểm thử	9

5. Checklist hoàn thành dự án

- ☐ Project chạy được từ đầu đến cuối bằng một lệnh duy nhất, môi trường tạo bằng UV
- ☐ Dữ liệu CV/JD đã xử lý sạch, cột Category được giữ riêng làm ground truth (không lộ vào input agent)
- ☐ Schema structured output áp dụng nhất quán cho mọi output của agent
- ☐ Ít nhất 3 subagent chấm điểm độc lập + 1 orchestrator tổng hợp
- ☐ Guardrail before-agent (redact thông tin cá nhân) và after-agent (chống hallucination kỹ năng) hoạt động và có test
- ☐ Graph LangGraph chạy được cho cả batch nhiều CV, có nhánh human-in-the-loop cho trường hợp borderline
- ☐ LLM Gateway có routing theo subagent, có cost tracking, có caching/fallback hoạt động thật khi xử lý batch
- ☐ Báo cáo evaluation có số liệu độ chính xác (precision@k theo Category) và điểm LLM-as-judge
- ☐ Giao diện demo cho phép upload batch CV, xem bảng xếp hạng đầy đủ, xuất CSV, và duyệt thủ công case borderline
- ☐ README có số liệu cụ thể, có thể trích trực tiếp vào resume